



Web Technologies

WET2VO

Teil 6: Templates und Routing

Medientechnik und -design | SS 2026

Template-Systeme

HTML- und PHP-Code sauber trennen

PHP & HTML bis jetzt



```
<?php
echo "Text";
function bla() {
?>
<h1>Überschrift</h1>
<?php
}
function moreHTML() {
    echo "<p>0h</p>";
    echo "<p>No</p>";
}
?>
```



Mit Templates



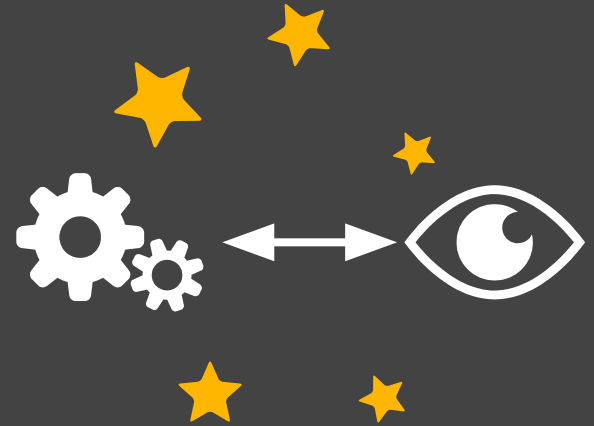
```
<?php
// Calculate stuff
$template->render("template1.latte");

// Calculate more Stuff
$template->render("template2.latte");
?>
```

```
<html>
  <head>
    <title>Site</title>
  </head>
  <body>
```

```
    <h1>Sup?</h1>
    <p>Hello World</p>
  </body>
</html>
```

Template-Systeme



Trennung von **Logik** und **Präsentation**

Vertreter: Latte, Twig, Smarty, mustache, Plates, Blade, Volt, ...

Latte

{latte}

PHP Template-Engine, aus dem Nette-Framework

Version 3.20.0 (13. Februar 2025)

<https://latte.nette.org/>



Installation

```
$ composer require latte/latte
```

Einbindung mittels Composer-Autoloader:

```
require __DIR__ . "/vendor/autoload.php";
```



Funktionsweise

```
$latte = new Engine();  
$latte->setLoader(  
    new FileLoader(__DIR__ . "/templates")  
);  
$latte->setCacheDirectory(__DIR__ . "/cache");  
  
$latte->render("message.latte", [  
    "name" => "John Doe",  
    "msg" => "I'm back baby!"  
]);
```

latte.php

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
    <title>Message</title>  
    <meta charset="utf-8">  
</head>  
<body>  
    <p>Name: {$name}</p>  
    <p>Message: {$msg}</p>  
</body>  
</html>
```

templates/message.latte



Arrays und Objekte

```
$array = ["John Doe", "male", 25];  
$latte→render("array-object.latte", [  
    "data1" ⇒ $array  
]);
```

```
$assocArray = [  
    "name" ⇒ "John Doe",  
    "details" ⇒ ["gender" ⇒ "male", "age" ⇒ 25]  
];  
$latte→render("array-object.latte", [  
    "data2" ⇒ $assocArray  
]);
```

```
$object = new Person("John Doe", "male", 25);  
$latte→render("array-object.latte", [  
    "data3" ⇒ $object  
]);
```

latte-array-object.php

```
<p>Name: {{$data1[0]}}</p>  
<p>Gender: {{$data1[1]}}</p>  
<p>Age: {{$data1[2]}}</p>
```

```
<p>Name: {{$data2["name"]}}</p>  
<p>Gender:  
{{$data2["details"]["gender"]}}</p>  
<p>Age:  
{{$data2["details"]["age"]}}</p>
```

```
<p>Name: {{$data3→name}}</p>  
<p>Gen.: {{$data3→gender}}</p>  
<p>Age: {{$data3→age}}</p>
```

templates/array-object.latte



Filter im Template

```
<p>Name: {$name|lower}</p>  
<p>Message: {$message|stripHtml|trim}</p>  
<p>Timestamp: {=now|date:'d.m.Y H:M'}</p>
```

Good to know:

Latte betreibt immer Escaping der Inhalte (Content Sanitation)
noescape deaktiviert das Auto-Escaping

Mehr zum Escaping und Filtering nächste Woche.

Tags: **{foreach}**



```
$array = [  
    "Rot",  
    "Grün",  
    "Blau"  
];  
$latte→render(  
    "loops.latte",  
    ["colors" ⇒ $array]  
);
```

PHP-Datei

```
<ul>  
{foreach $colors as $color}  
    <li>{$color}</li>  
{/foreach}  
</ul>
```

Template

```
<ul>  
    <li>Rot</li>  
    <li>Grün</li>  
    <li>Blau</li>  
</ul>
```

Output

n:attributes: n:foreach



```
$array = [  
    "Rot",  
    "Grün",  
    "Blau"  
];  
$latte→render(  
    "loops.latte",  
    ["colors" ⇒ $array]  
);
```

PHP-Datei

```
<ul>  
    <li n:foreach="$colors as  
        $color">{$color}</li>  
</ul>
```

Template

```
<ul>  
    <li>Rot</li>  
    <li>Grün</li>  
    <li>Blau</li>  
</ul>
```

Output

Tags: { for }



```
$latte→render(  
    "loops.latte"  
);
```

PHP-Datei

```
<ul>  
  {for $i = 1; $i ≤ 3; $i++}  
    <li>{$i}</li>  
  {/endfor}  
</ul>
```

Template

```
<ul>  
  <li>1</li>  
  <li>2</li>  
  <li>3</li>  
</ul>
```

Output

n:attributes: n:for



```
$latte→render(  
    "loops.latte"  
);
```

PHP-Datei

```
<ul>  
    <li n:for="$i = 1; $i ≤  
3; $i++">{$i}</li>  
</ul>
```

Template

```
<ul>  
    <li>1</li>  
    <li>2</li>  
    <li>3</li>  
</ul>
```

Output

Tags: **{if}** , **{elseif}** , **{else}**



```
{if $gender == "male"}  
<p>Welcome Sir!</p>  
{elseif gender == "female"}  
<p>Welcome Madam!</p>  
{else}  
<p>Welcome non-binary Person!</p>  
{/if}
```

Beispiel 1 (if/elseif/else)

```
{ifset $name}  
<p>Welcome {$name}!</p>  
{/ifset}  
  
<p n:ifset="$name">  
  Welcome {$name}!</p>
```

Beispiel 2 (ifset)

Tags: **{include}**



```
<!DOCTYPE html>
<html>
<head>
  <title>Hello World</title>
  <meta charset="utf-8">
</head>
<body>
  {include "body.latte", "heading" =>
    "Welcome to the site!"}
</body>
</html>
```

main.latte

```
<h1>{$heading}</h1>
<p>Hello World!</p>
```

body.latte



Wie viel **Logik** darf ins Template?

LaTeX (logic-full)

```
{\foreach \color in {red,blue,green}}  
  <li>{\color}\</li>  
{/foreach}
```

Mustache (logic-less)

```
{{# colors }}  
  * {{ . }}  
{{/ colors }}
```

Routing

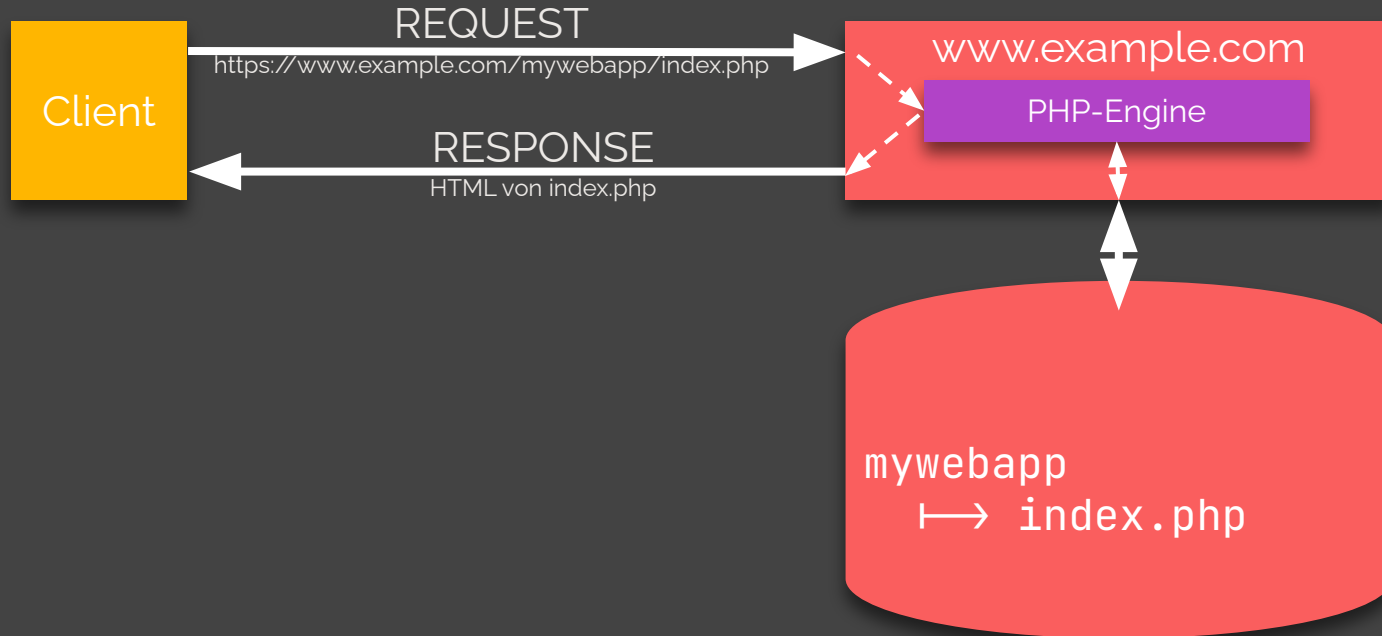
HTTP-Requests je nach Pfad weiterleiten

Routing?

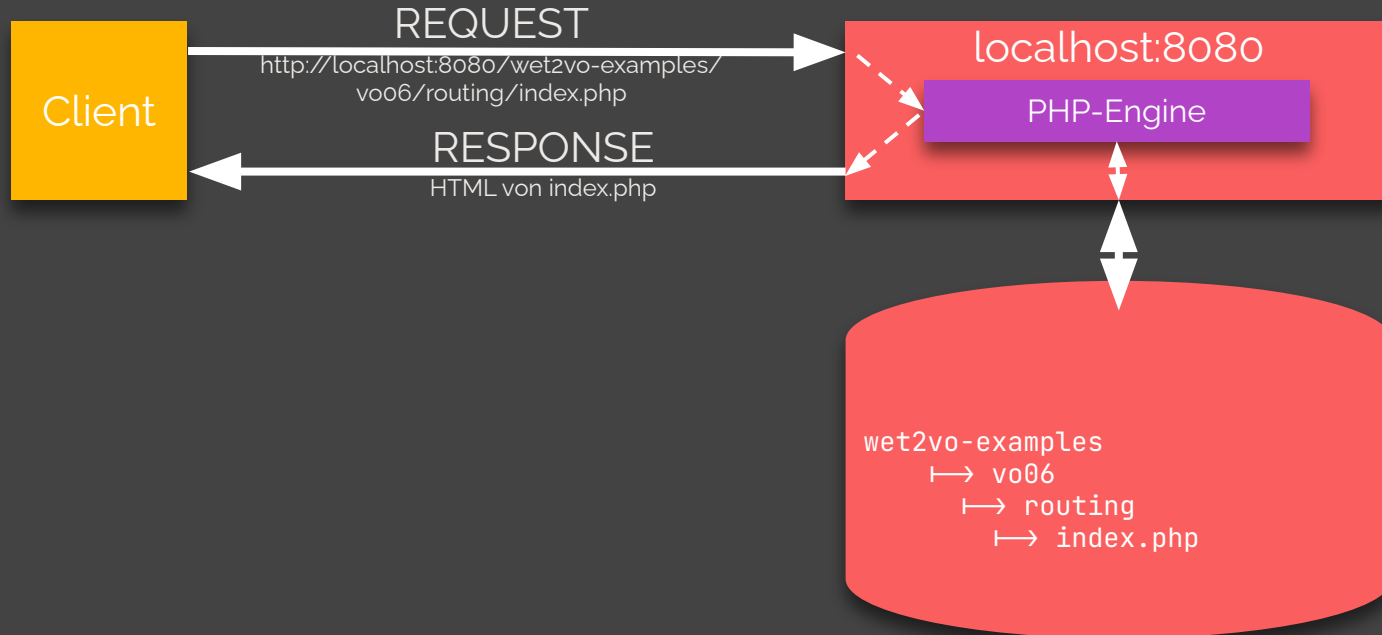
Ein Mechanismus, bei dem **HTTP-Requests** (d.h. der Aufruf bestimmter URLs) an **Code**, der sich um deren **Abarbeitung** kümmert, **weitergeleitet** werden.



PHP: **Webserver** macht



PHP: Webserver macht





Einfach, oder?

Seite mit User-Management

Etwa so:

Hauptseite

<https://www.example.com/mywebapp/index.php>

Registrierung

<https://www.example.com/mywebapp/register.php>

Login

<https://www.example.com/mywebapp/login.php>

Logout

<https://www.example.com/mywebapp/logout.php>

Schönere URLs?

Lieber so:

Hauptseite

<https://www.example.com/mywebapp>

Registrierung

<https://www.example.com/mywebapp/account/register>

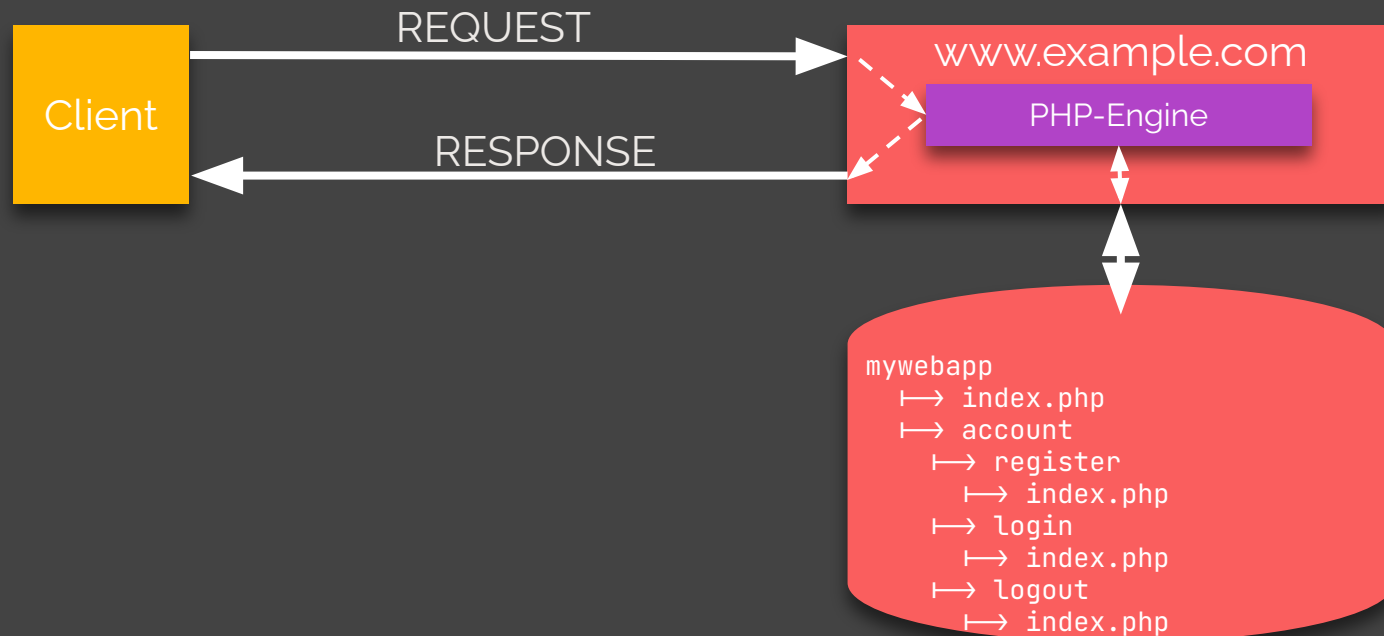
Login

<https://www.example.com/mywebapp/account/login>

Logout

<https://www.example.com/mywebapp/account/logout>

Ein Versuch





Aber ...

- ... was passiert bei hunderten URLs?
- ... was, wenn Parameter in der URL nicht bekannt sind? z.B.
`https://www.example.com/mywebapp/product/4`

Ein besserer Mechanismus muss her: **Routing!**

Front Controller r



Alle Requests laufen an einer zentralen Stelle zusammen und werden dort behandelt (Design Pattern).

In PHP in einer Datei: `index.php`

URL-Rewriting am Server



.htaccess:

```
<IfModule mod_rewrite.c>
```

```
RewriteEngine On
```

```
RewriteCond %{REQUEST_URI}::$1 ^(/.+)/(.*):~2$
```

```
RewriteRule ^(.*) - [E=BASE:%1]
```

```
RewriteCond %{REQUEST_FILENAME} !-f
```

```
RewriteRule ^ index.php [QSA,L]
```

```
</IfModule>
```

Basis-Verzeichnis
ermitteln (falls
Applikation nicht im
Root des Webserver)

Wenn die Datei nicht
existiert, leite auf
index.php weiter



Die aktuelle **Route** ermitteln

HTTP-Methode

```
$method =  
$_SERVER["REQUEST_METHOD"];
```

z.B. GET oder POST

Request-Pfad

```
$path =  
$_SERVER["REQUEST_URI"];
```

z.B. /mywebapp oder
/wet2vo-examples/vo06/routing

Problem: Unterordner



Request-Pfade:

`/mywebapp/` und `/mywebapp/login`
`/wet2vo-examples/vo06/routing/` und
`/wet2vo-examples/vo06/routing/other`

Applikation liegt nicht im Root-Ordner des Webserver, sondern in einem Unterordner. Dieser soll aber nicht Teil der Route sein.

Lösung: Basispfad

```
$basePath = "/wet2vo-examples/vo06/routing";
```

Route ohne Basispfad



Die finale Route:

```
if (!empty($basePath)) {  
    $path = substr($path, strlen($basePath));  
}
```

```
$route = "$method $path";
```

```
GET / für http://localhost:8080/wet2vo-examples/vo06/routing/
```

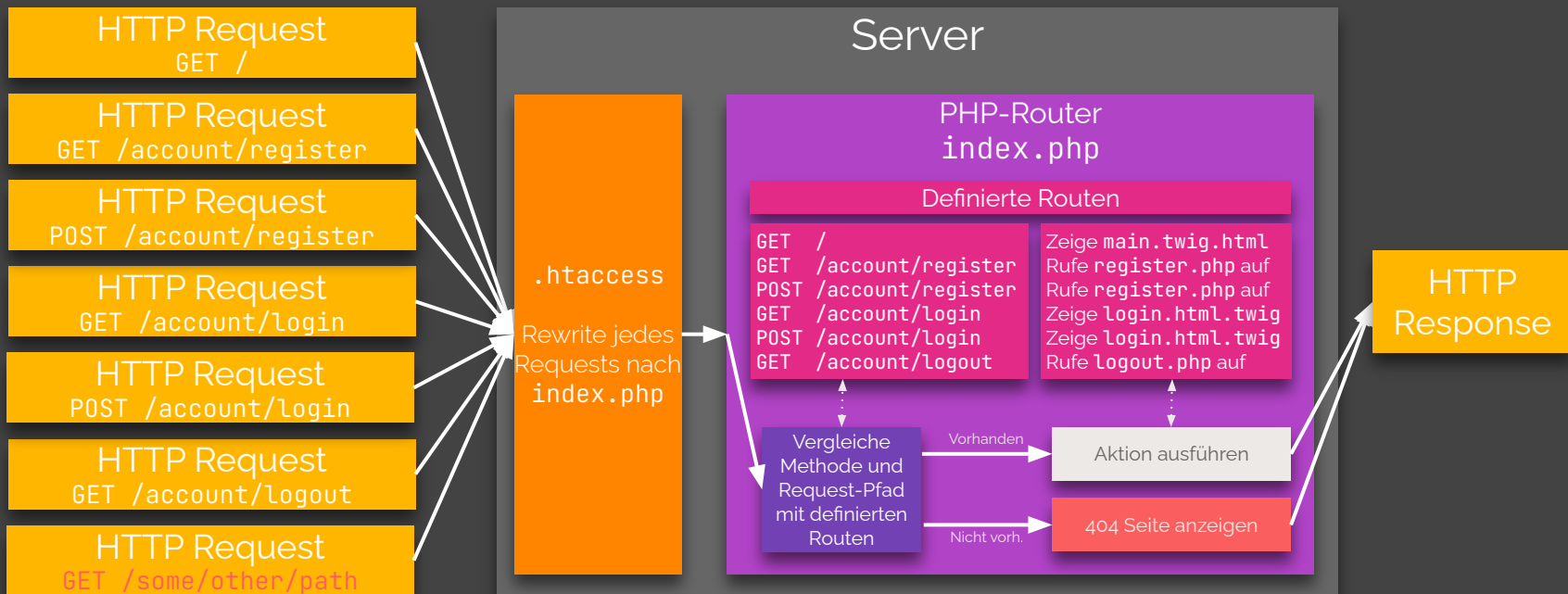
```
GET /other für http://localhost:8080/wet2vo-examples/vo06/routing/other
```

Aktionen für Routen



```
switch ($route) {  
    case "GET /":  
        echo "Here's the root page";  
        break;  
    case "GET /other":  
        echo "Here's the other page.";  
        break;  
    default:  
        http_response_code(404);  
        echo "Page not found!";  
}
```

Routing Prinzip



3

fhooe/router

Einfaches Routing



Installation

```
$ composer require fhoee/router
```

Einbindung mittels Composer-Autoloader in `index.php`:

```
require __DIR__ . "/vendor/autoload.php";
```

`.htaccess` ins selbe Verzeichnis



5 Schritte

1. Lege ein **Router-Objekt** an.
2. Optional: Setze einen **Basispfad**, falls index.php in einem Unterverzeichnis liegt.
3. Definiere alle gewünschten **Routen** und die dazugehörigen Aktionen, die eine Response erzeugen.
4. Definiere eine **404-Aktion**, falls ein Pfad aufgerufen wird, für den es keine Route gibt.
5. **Starte** den Routing-Mechanismus.

Schritte 1 und 2



```
<?php
```

```
declare(strict_types=1);
```

```
use Fhooe\Router\Router;
```

```
require __DIR__ . "/vendor/autoload.php";
```

```
// 1. Create the Router object
```

```
$router = new Router();
```

```
// 2. Optional: set the base path if the application is in a subdirectory
```

```
$router->basePath = "/wet2vo-examples/vo06/fhooe-router";
```

use-Statement verkürzt den Namespace.
Sonst: `new Fhooe\Router\Router();`

Pfad muss mit "/"
beginnen und darf am
Ende keinen aufweisen.

Schritte 3 bis 5



```
// 3. Define all the needed routes
$router->get("/", function () {
    require "views/main.php";
});

$router->get("/form", function () {
    require "views/form.php";
});

$router->post("/form", function () {
    require "views/form.php";
});
```

Route wird mit Methode `get()` oder `post()` angelegt. Parameter sind der Pfad und ein Callback mit der Aktion.

```
// 4. Define a 404 callback
$router->set404Callback(function () {
    require "views/404.html";
});
```

404-Aktion ist nur ein Callback mit einer Aktion.

```
// 5. Start the routing mechanism
$router->run();
```

Vergleicht die hinterlegten Routen und führt die Aktion oder 404 aus.

Routen im Detail



Webserver: **localhost:8080**

Basispfad: **/wet2vo-examples/vo06/fhooe-router**

```
$router→get("/", function () {  
    require "views/main.php";  
});
```

→ GET **http://localhost:8080/wet2vo-examples/vo06/fhooe-router/**

```
$router→get("/form", function () {  
    require "views/form.php";  
});
```

→ GET **http://localhost:8080/wet2vo-examples/vo06/fhooe-router/form**

```
$router→post("/form", function () {  
    require "views/form.php";  
});
```

→ POST **http://localhost:8080/wet2vo-examples/vo06/fhooe-router/form**

Ausgabe: **main.php**



```
<?php

global $router;
?>

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Main Page</title>
</head>
<body>
  <h1>Main</h1>
  <p>This is the main page!</p>
  <p>Why not also try the <a href="<?= $router->urlFor("/form") ?>">form</a>?</p>
</body>
</html>
```

Ausgabe: **form.php**



```
<?php
global $router;
?>

<!DOCTYPE html>
<html lang="en">
...
<body>
  <h1>Form</h1>
  <p>This is the form page!</p>
  <form action="<?= $router->urlFor("/form") ?>" method="post">
    <label for="message">Message:</label>
    <input type="text" name="msg" id="message">
    <button type="submit">Send Message</button>
  </form>

  <?php
  if (isset($_POST["msg"])) {
    echo "<p>Message: {$_POST["msg"]}</p>";
  }
  ?>

  <p>Why not also try the <a href="<?= $router->urlFor("/") ?>">main page</a>?</p>
</body>
</html>
```

Ausgabe: 404.html



```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>404</title>
</head>
<body>
  <h1>404</h1>
  <p>File not found.</p>
</body>
</html>
```




Weitere Funktionalitäten

Basispfad abfragen

```
$basePath =  
  $router→basePath;
```

Redirect auf URL

```
$router→redirect(  
  "https://www.fh-ooe.at/"  
);
```

Redirect auf Route

```
$router→redirectTo("/form");
```

fhooe/router-skeleton

Beispielprojekt für fhooe/router



Installation

```
$ composer create-project fhoee/router-skeleton path
```

path muss ein nicht existierendes Verzeichnis sein.

```
$ composer create-project fhoee/router-skeleton fhoee-router-skeleton
```

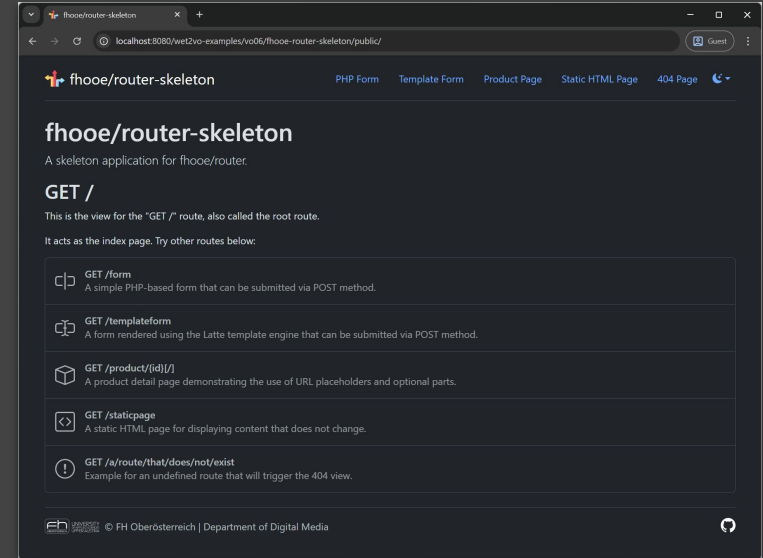
```
$ composer create-project fhoee/router-skeleton ue06
```

```
$ composer create-project fhoee/router-skeleton myproject
```



Ein **Blick** ins Projekt

fhoove/router-skeleton ist ein **Basisprojekt** mit Latte-Templates und Logging.



Komplexere Router

Wenn's mal mehr sein soll



bramus/router

```
$ composer require bramus/router
```

- HTTP-Methoden GET, POST, PUT, DELETE, OPTIONS, PATCH und HEAD.
- Dynamische Routing-Muster (Pfade) über Regular Expressions.
- Untermuster für Routen, bei denen Teile optional gemacht werden können.
- Middleware Support (Code, der vor oder nach dem Routing ausgeführt werden kann).

bramus/router

A lightweight and simple object oriented PHP Router



 20
Contributors

 3k
Used by

 1k
Stars

 238
Forks



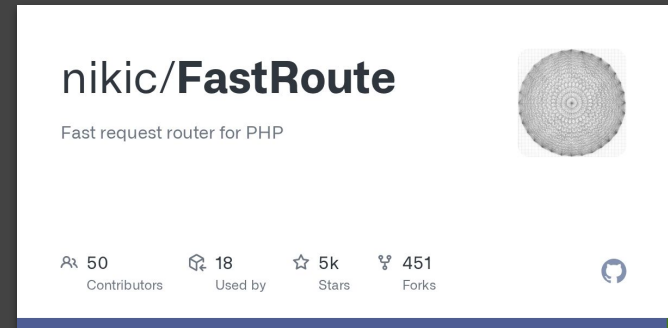


nikic/fast-route

e

```
$ composer require nikic/fast-route
```

- Fokus auf Geschwindigkeit.
- Schnell durch kombinierte Regular Expressions.
- Gut für große Projekte.
- In vielen Open-Source-Projekten eingesetzt (z.B. Slim Framework)





league/route

\$ composer require league/route

- Standardkonformität durch PSRs.
- PSR-1, PSR-2, PSR-4, PSR-7, PSR-11 und PSR-15.
- Baut auf nikic/fast-route auf.
- Benutzer*innenfreundliche Weiterentwicklung.

**thephpleague/
route**



Fast PSR-7 based routing and dispatch component including PSR-15 middleware, built on top of FastRoute.

 42
Contributors

 5
Issues

 3
Discussions

 662
Stars

 127
Forks





Nächstes Mal in WET2VO

Formularverarbeitung

Credits



- S. 6: Latte Logo: <https://latte.nette.org/>
- S. 46: bramus/router GitHub Social Image: <https://github.com/bramus/router>
- S. 47: nikic/FastRout GitHub Social Image: <https://github.com/nikic/fastroute>
- S. 48: thephpleague/route GitHub Social Image: <https://github.com/thephpleague/route>